计算机学院实验报告

实验题目：光线追踪渲染图像 2		学号：202200130077
日期：2024. 6. 4	班级：智能	姓名：薛润泽
Email：		

实验目的：

1、IntersectP(const Ray& ray, const Vector3f& invDir,
const std::array<int, 3>& dirIsNeg) in the Bounds3.hpp: 这个函数的作用是判断包围盒 BoundingBox 与光线是否相交，你需要按照课程介绍的算法实现求交过程。

2、getIntersection(BVHBuildNode* node, const Ray ray)in BVH.cpp: 建立 BVH 之后，我们可以用它加速求交过程。该过程递归进行，你将在其中调用你实现的 Bounds3::IntersectP.

实验环境介绍：

Ubuntu

解决问题的主要思路：

本实验的主要思路如下：

1. 学习如何判断 AABB 包围盒与光线是否有交点，在本次实验中完成包围盒与光线判断求交代码。
2. 学习 BVH 树形结构，使用 BVH 结构分割物体空间三角形，建立不同层次包围盒。在本次实验中完成在 BVH 树形结构中递归搜索光线与包围盒交点。

实验步骤与实验结果：

1. AABB 包围盒与光线 Ray 判断求交算法

判断 AABB 包围盒与光线 Ray 是否有交点可以使用类似 Liang-barsky 裁剪算法的思想。对于两条平行直线来讲，如果存在一条与其不平行且共面的有向直线，那么这条有向直线一定可以交这两条直线与两个不同点。因为该直线有方向，从直线动态延伸的角度来说，这两个交点的产生有时间上先后顺序。一个 AABB 包围盒有三个维度，包裹六个面对于每个维度，我们求出光线在 x，y，z 三个维度上的光线进入时间和光线离开时间。对于怎样算是“进入了包围盒”，我们定义 x，y，z 三个维度进入时间的最大值为进入包围盒的进入时间，三个维度离开包围盒的最小值为离开包围盒的离开时间。如果进入时间小于离开时间，并且离开时间是一个正值，那么认为此光线进入过包围盒。如果是其他情况，则认为光线和包围盒没有交点。

2、BVH

包围盒可以有两种创建方式。第一种是对空间进行划分，划分空间包围盒，这种包围盒的优点是划分的空间彼此无交集，且所有划分空间的并就是全集，缺点是可能会出现物体在多个包围盒中的情况。第二种是对空间中的物体进行划分，优点是物体会唯一出现在某个包围盒中（如果一个包围盒完全包含另一个包围盒，当然这个物体也会出现在更大的这个包围盒中）。缺点是划分空间之间可能会有交集。BVH 选择了第二种包围盒创建方法。首先选择划分平面，因为目的包围盒是 AABB 包围盒，是轴对称的，所以这个平面是平行于 x-y 平面或者平行于 y-z 平面或者平行于 x-z 平面的。通常选用空间中物体三个坐标中，极差最大的那个坐标对应的平面作为划分平面（这样也可以降低 BVH 的树高度）。然后划分位置选择中位点物体的所在位置，意味着在本位置划分后可以将两侧划分出相同个物体的，两个包围盒内的物体总数接近相同。

不断进行划分，直到包围盒中的物体数目低于我们的阈值，则停止划分。

在该实验中，BVH 已经完成了建造，我们需要实现的代码是如何使用递归方法来求得光线 Ray 何 BVH 的交点。

下面是代码实现：

//IntersectP函数，用于计算是否包围盒与光线是否有交点

```
inline bool Bounds3::IntersectP(const Ray& ray, const
Vector3f& invDir,
                                const std::array<int, 3>& dirIsNeg)
const
{
    // invDir: ray direction(x,y,z),
invDir=(1.0/x,1.0/y,1.0/z), use this because Multiply is
faster that Division
```

```
    // dirIsNeg: ray direction(x,y,z),
dirIsNeg=[int(x>0),int(y>0),int(z>0)], use this to simplify
your logic
    // TODO test if ray bound intersects
    double tx_min = (pMin.x - ray.origin.x) * invDir.x;
    double tx_max = (pMax.x - ray.origin.x) * invDir.x;
    if(dirIsNeg[0] <= 0) std::swap(tx_min, tx_max);

    double ty_min = (pMin.y - ray.origin.y) * invDir.y;
    double ty_max = (pMax.y - ray.origin.y) * invDir.y;
    if(dirIsNeg[1] <= 0) std::swap(ty_min, ty_max);

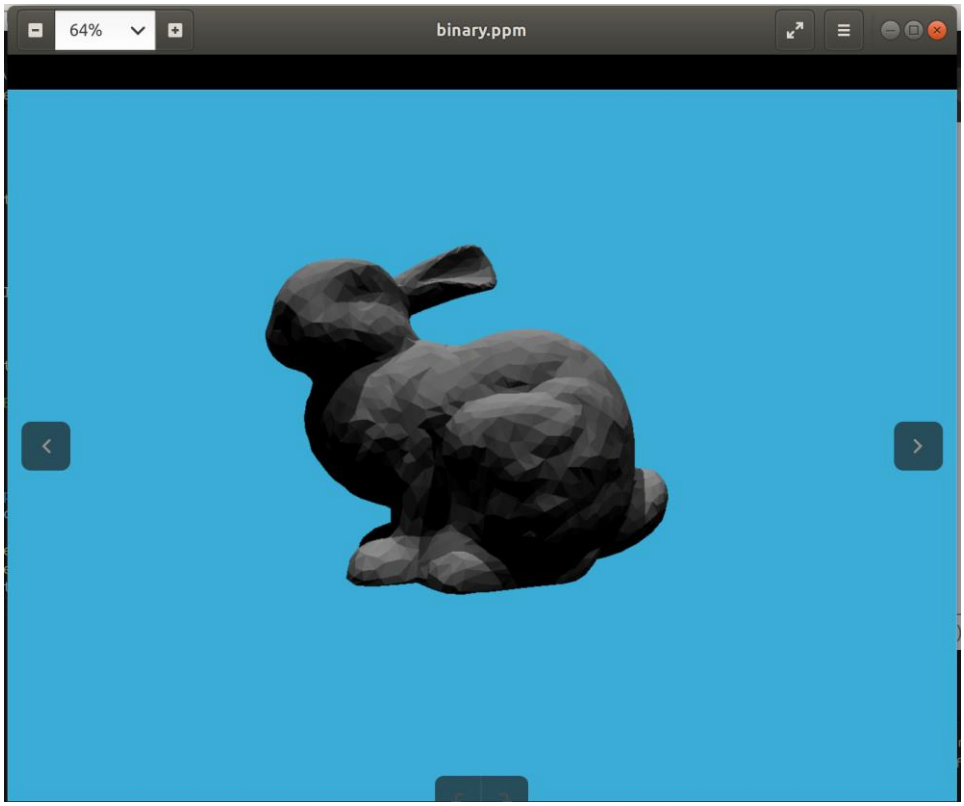
    double tz_min = (pMin.z - ray.origin.z) * invDir.z;
    double tz_max = (pMax.z - ray.origin.z) * invDir.z;
    if(dirIsNeg[2] <= 0) std::swap(tz_min, tz_max);
    double t_enter = std::max(tx_min, std::max(ty_min,
tz_min));
    double t_exit = std::min(tx_max, std::min(ty_max,
tz_max));
    if(t_enter < t_exit && t_exit >= 0) return 1;
    else return 0;

}
```

// getIntersection函数，用来递归查询光线与BVH树中哪一个包围盒有交点

```
Intersection BVHAccel::getIntersection(BVHBuildNode* node,
const Ray& ray) const
{
    // TODO Traverse the BVH to find intersection
    Intersection insect;
    if(node == nullptr || node->bounds.IntersectP(ray,
ray.direction_inv, std::array<int,3>{ray.direction.x >
0,ray.direction.y > 0,ray.direction.z > 0}) == 0){
        return insect;
    }
    if(node->left == nullptr && node->right == nullptr){
        return node->object->getIntersection(ray);
    }
    Intersection hit1 = getIntersection(node->left, ray);
    Intersection hit2 = getIntersection(node->right, ray);
    if(hit1.distance < hit2.distance) return hit1;
    else return hit2;
}
```

实验效果：



实验中存在的问题及解决：

问题 1：在判断光线与包围盒求交中，为什么只要入射时间小于出射时间并且出射时间大于零就可以确定该光线与包围盒有交点？如果入射时间是一个负值说明什么？

答 1：我们允许入射时间是一个负数，因为入射时间是一个负数的话，说明光线发出点在包围盒的中间，这种情况是合法的。

问题 2：为什么使用 BVH 递归求光线与包围盒交点可以起到加速作用？

答 2：因为在递归求交点的过程中避免了遍历全局三角形。如果 BVH 的一个节点代表的包围盒已经不再与光线有交点，那么其 child 代表的包围盒也不会与光线有交点，所以可以直接省略其计算过程。